9.10 SUMMARY

Computers frequently contain valuable and confidential data, including tax returns, credit card numbers, business plans, trade secrets, and much more. The owners of these computers are usually quite keen on having them remain private and not tampered with, which rapidly leads to the requirement that operating systems must provide good security. Security comprises confidentiality, integrity, and availability. To develop secure systems, we consistently apply security principles. The structure of an operating system matters for its security properties, as some designs (e.g., monolithic systems) make it difficult to apply principles such as the Principle Of Least Authority. In general, the security of a system is inversely proportional to the size of the trusted computing base.

A fundamental component of security for operating systems concerns access control to resources. Access rights to information can be modeled as a big matrix, with the rows being the domains (users) and the columns being the objects (e.g., files). Each cell specifies the access rights of the domain to the object. Since the matrix is sparse, it can be stored by row, which becomes a capability list saying what that domain can do, or by column, in which case it becomes an access control list telling who can access the object and how. Using formal modeling techniques, information flow in a system can be modeled and limited. However, sometimes it can still leak out using covert channels, such as modulating CPU usage.

Security properties that rest on strong proofs and formalizations go beyond the modeling of information flow and include cryptography. Cryptographic schemes can be categorized as secret key or public key. A secret-key method requires the communicating parties to exchange a secret key in advance, using some out-of-band mechanism. Public-key cryptography does not require secretly exchanging a key in advance, but it is much slower in use. Sometimes it is necessary to prove the authenticity of digital information, in which case cryptographic hashes, digital signatures, and certificates signed by a trusted certification authority can be used.

In any secure system, users must be authenticated. This can be done by something the user knows, something the user has, or something the user is (biometrics). Two-factor identification, such as an iris scan and a password, can be used to enhance security.

Many kinds of bugs in software can be exploited to take over programs and systems. These include buffer overflow vulnerabilities, format-string bugs, use-after-free, type confusion bugs, null pointer dereferences, double frees, integer overflows, and several others. They enable a variety of attacks, nowadays frequently reusing the original program code to implement the malicious behavior. To deal with such vulnerabilities, vendors deploy defenses such as stack canaries, data execution prevention, and address-space layout randomization.

Insiders, such as company employees, can defeat system security in different ways. These include logic bombs set to go off on some future date, trap doors to allow the insider unauthorized access later, and login spoofing.

Unfortunately, software bugs are no longer our only concern and hardware is often vulnerable too. Cache side channels and ancient execution issues are some of the issues on which attackers can build to launch an attack.

To protect itself from compromise, an operating system may take additional measures. For instance, by restricting the control flow in a program with the help of control-flow integrity, secure boot processes, randomizing the address space at fine granularity, restricting trusting only drivers that are signed by a trusted party—or one of the other ways to harden the operating system.

PROBLEMS

1. Confidentiality, integrity, and availability are three components of security. Describe an application that requires integrity and availability but not confidentiality, an application that requires confidentiality and integrity but not (high) availability, and an application that requires confidentiality, integrity, and availability.
2. One of the techniques to build a secure operating system is to minimize the size of the TCB. Which of the following functions needs to be implemented inside the TCB and which can be implemented outside TCB: (a) Process context switch; (b) Read a file from disk; (c) Add more swapping space; (d) Listen to music; (e) Get the GPS coordinates of a smartphone.
3. What is a covert channel? What is the basic requirement for a covert channel to exist?
4. In a full access-control matrix, the rows are for domains and the columns are for objects. What happens if some object is needed in two domains?
5. Suppose that a system has 1000 objects and 100 domains at some time. 1% of the objects are accessible (some combination of *r*, *w* and *x*) in all domains, 10% are accessible in two domains, and the remaining 89% are accessible in only one domain. Suppose one unit of space is required to store an access right (some combination of *r*, *w*, *x*), object ID, or a domain ID. How much space is needed to store the full protection matrix, protection matrix as ACL, and protection matrix as capability list?
6. Explain which implementation of the protection matrix is more suitable for the following operations:

   (a) Granting read access to a file for all users.
   (b) Revoking write access to a file from all users.
   (c) Granting write access to a file to John, Lisa, Christie, and Jeff.
   (d) Revoking execute access to a file from Jana, Mike, Molly, and Shane.
7. Two different protection mechanisms that we have discussed are capabilities and access-control lists. For each of the following protection problems, tell which of these mechanisms can be used.

   (a) Ken wants his files readable by everyone except his office mate.
   (b) Mitch and Steve want to share some secret files.
   (c) Linda wants some of her files to be public.

8. Represent the ownerships and permissions shown in this UNIX directory listing as a protection matrix. (*Note*: *asw* is a member of two groups: *users* and *devel*; *gmw* is a member only of *users*.) Treat each of the two users and two groups as a domain, so that the matrix has four rows (one per domain) and four columns (one per file).

–rw–r––r––	2	gmw	users	908	May 26 16:45	PPP–Notes
–rwx r–x r–x	1	asw	devel	432	May 13 12:35	prog1
–rw–rw––––	1	asw	users	50094	May 30 17:51	project.t
–rw–r–––––	1	asw	devel	13124	May 31 14:30	splash.gif

9. Express the permissions shown in the directory listing of the previous problem as access-control lists.

10. Modify the ACL from the previous problem for one file to grant or deny an access that cannot be expressed using the UNIX *rwx* system. Explain this modification.

11. Suppose there are three security levels, 1, 2, and 3. Objects *A* and *B* are at level 1, *C* and *D* are at level 2, and *E* and *F* are at level 3. Processes 1 and 2 are at level 1, 3 and 4 are at level 2, and 5 and 6 are at level 3. For each of the following operations, specify whether they are permissible under Bell-LaPadula model, Biba model, or both.

(a) Process 1 writes object *D*
(b) Process 4 reads object *A*
(c) Process 3 reads object *C*
(d) Process 3 writes object *C*
(e) Process 2 reads object *D*
(f) Process 5 writes object *F*
(g) Process 6 reads object *E*
(h) Process 4 write object *E*
(i) Process 3 reads object *F*

12. In the Amoeba scheme for protecting capabilities, a user can ask the server to produce a new capability with fewer rights, which can then be given to a friend. What happens if the friend asks the server to remove even more rights so that the friend can give it to someone else?

13. In Fig. 9-11, there is no arrow from object *2* to process *A*. Would such an arrow be allowed? If not, what rule would it violate?

14. If process-to-process messages were allowed in Fig. 9-11, what rules would apply to them? For process *B* in particular, to which processes could it send messages and which not?

15. Break the following monoalphabetic cipher. The plaintext, consisting of letters only, is a well-known excerpt from a poem by Lewis Carroll.

hur iby eci iulylyt zy hur irc
iulylyt elhu cxx uli oltuh
ur nln uli jrkd grih hz ocvr
hur glxxzei iozzhu cyn gkltuh
cyn huli eci znn grqcbir lh eci
hur olnnxr zp hur yltuh

16. Consider a secret-key cipher that has a 26 × 26 matrix with the columns headed by *ABC ... Z* and the rows also named *ABC ... Z*. Plaintext is encrypted two characters at a time. The first character is the column; the second is the row. The cell formed by the intersection of the row and column contains two ciphertext characters. What constraint must the matrix adhere to and how many keys are there?

17. Consider the following way to encrypt a file. The encryption algorithm uses two *n*-byte arrays, *A* and *B*. The first *n* bytes are read from the file into *A*. Then *A*[0] is copied to *B*[*i*], *A*[1] is copied to *B*[*j*], *A*[2] is copied to *B*[*k*], etc. After all *n* bytes are copied to the *B* array, that array is written to the output file and *n* more bytes are read into *A*. This procedure continues until the entire file has been encrypted. Note that here encryption is not being done by replacing characters with other ones, but by changing their order. How many keys have to be tried to exhaustively search the key space? Give an advantage of this scheme over a monoalphabetic substitution cipher.

18. Secret-key cryptography is more efficient than public-key cryptography, but requires the sender and receiver to agree on a key in advance. Suppose that the sender and receiver have never met, but there exists a trusted third party that shares a secret key with the sender and also shares a (different) secret key with the receiver. How can the sender and receiver establish a new shared secret key under these circumstances?

19. Give a simple example of a mathematical function that to a first approximation will do as a one-way function.

20. Suppose that two strangers *A* and *B* want to communicate with each other using secret-key cryptography, but do not share a key. Suppose both of them trust a third party *C* whose public key is well known. How can the two strangers establish a new shared secret key under these circumstances?

21. Internet cafes are businesses where tourists away from home can rent a computer for an hour or two to do business that needs a computer. Describe a way to produce signed documents from one using a smart card (assume that all the computers are equipped with smart-card readers). Is your scheme secure?

22. Not having the computer echo the password is safer than having it echo an asterisk for each character typed, since the latter discloses the password length to anyone nearby who can see the screen. Assuming that passwords consist of upper and lowercase letters and digits only, and that passwords must be a minimum of five characters and a maximum of eight characters, how much safer is not displaying anything?

23. After getting your degree, you apply for a job as director of a large university computer center that has just put its ancient mainframe system out to pasture and switched over to a large LAN server running UNIX. You get the job. Fifteen minutes after you start work, your assistant bursts into your office screaming: "Some students have discovered the algorithm we use for encrypting passwords and posted it on the Internet." What should you do?

24. The Morris-Thompson protection scheme with *n*-bit random numbers (salt) was designed to make it difficult for an intruder to discover a large number of passwords by encrypting common strings in advance. Does the scheme also offer protection against a student user who is trying to guess the superuser password on his machine? Assume the password file is available for reading.

25. Explain how the UNIX password mechanism is different from encryption.

26. Suppose the password file of a system is available to a cracker. How much extra time does the cracker need to crack all passwords if the system is using the Morris-Thompson protection scheme with n-bit salt versus if the system is not using this scheme?

27. Name three characteristics that a good biometric indicator must have in order to be useful as a login authenticator.

28. Name three biometric characteristics that would not be good to use for authentication.

29. Authentication mechanisms are divided into three categories: Something the user knows, something the user has, and something the user is. Imagine an authentication system that uses a combination of these three categories. For example, it first asks the user to enter a login and password, then insert a plastic card (with magnetic strip) and enter a PIN, and finally provide fingerprints. Can you think of two drawbacks of this design?

30. A computer science department has a large collection of UNIX machines on its local network. Users on any machine can issue a command of the form

```
rexec machine4 who
```

and have the command executed on *machine4*, without having the user log in on the remote machine. This feature is implemented by having the user's kernel send the command and his UID to the remote machine. Is this scheme secure if the kernels are all trustworthy? What if some of the machines are students' personal computers, with no protection? Assume the network cannot be tapped.

31. What property does the implementation of passwords in UNIX have in common with Lamport's scheme for logging in over an insecure network?

32. Is there any feasible way to use the MMU hardware to prevent the kind of overflow attack shown in Fig. 9-17? Explain why or why not.

33. Describe how stack canaries work and how they can be circumvented by the attackers.

34. The TOCTOU attack exploits a race condition between the attacker and the victim. One way to prevent race conditions is make file system accesses transactions. Explain how this approach might work and what problems might arise?

35. When a file is removed, its blocks are generally put back on the free list, but they are not erased. Do you think it would be a good idea to have the operating system erase each block before releasing it? Consider both security and performance factors in your answer, and explain the effect of each.

36. To verify that an downloaded driver has been signed by a trusted vendor, the drivervendor may include a certificate signed by a trusted third party that contains its public key. However, to read the certificate, the user needs the trusted third party's public key. This could be provided by a trusted fourth party, but then the user needs that public key. It appears that there is no way to bootstrap the verification system, yet existing browsers use it. How could it work?

37. In Sec. 9.5.4, we saw that type confusion bugs in C++ are caused by bugs when statically casting a parent type to the wrong child type. Besides the

```
static_cast
```

construct, C++ also supports dynamics casts, using the

```
dynamic_cast
```

construct. Dynamic casts are enforced at run time with an explicit type check and are therefore ensure type safety. Why would programmers not simply use dynamic casts everywhere and get rid of most type confusion bugs altogether?

38. One major problem with format string vulnerabilities is the "%n" formatting indicator which is hardly used by anyone. For this reason, many C libraries no longer support "%n" by default. Will this solve the problem of format string vulnerabilities?

39. To mitigate cache side channels, we want to partition the cache such that two processes always use different parts of the cache. Unfortunately, hardware support for such partitioning is often lacking. What could the operating system do to provide such partitioning?

40. In Sec. 9.6.3, we mentioned that we can stop many of the Spectre problems by inserting so-called *fence* instructions which stop speculation across that instruction. There are many other and much more complicated other mitigations also, but hey, let us at least stick a fence instruction after every "if" condition. Doing so is very simple and would get rid of a large number of vulerabilities. Explain why this is not a good idea.

41. In Sec. 9.7.3 we described login spoofing as an attack in which the attacker starts a program on a computer that displays a fake login screen on a computer. This would typically be used in a room full of computers at a university that students could use for assignments. When the student sits down and enters a login name and password, the fake program sends them off to its owner and then exits. The second time the student tries to login in it works and the students thinks there must of been a typing error the first time. Devise a way in which the operating system could defeat this kind of spoofing attack.

42. Write a pair of programs, in C or as shell scripts, to send and receive a message by a covert channel on a UNIX system. (*Hint*: A permission bit can be seen even when a file is otherwise inaccessible, and the *sleep* command or system call is guaranteed to delay for a fixed time, set by its argument.) Measure the data rate on an idle system. Then create an artificially heavy load by starting up numerous different background processes and measure the data rate again.

43. Several UNIX systems use the DES algorithm for encrypting passwords. These systems typically apply DES 25 times in a row to obtain the encrypted password. Download an implementation of DES from the Internet and write a program that encrypts a password and checks if a password is valid for such a system. Generate a list of 10 encrypted passwords using the Morris-Thomson protection scheme. Use 16-bit salt for your program.

44. Suppose a system uses ACLs to maintain its protection matrix. Write a set of management functions to manage the ACLs when (1) a new object is created; (2) an object is deleted; (3) a new domain is created; (4) a domain is deleted; (5) new access rights (a combination of r, w, x) are granted to a domain to access an object; (6) existing access rights of a domain to access an object are revoked; (7) new access rights are

granted to all domains to access an object; (8) access rights to access an object are revoked from all domains.

45. Implement the program code outlined in Sec. 9.5.1 to see what happens when there is buffer overflow. Experiment with different string sizes.

46. In this chapter,, we discussed covert channels. In this exercise, you are to run an experiment to determine the bandwidth of one such channel, namely file locking. You are to write two programs that will try to communicate in a sneaky way, the sender and the receiver. They are to run on the same computer. The communicate by using a file called *lockfile*. Both programs can read and lock *lockfile* but cannot write it. Sender transmits a 0 by leaving *lockfile* unlocked. It sends a 1 by locking it. For simplicity, assume time is discrete in units of Δt, with one bit transmitted every Δt. The two programs are assumed to be synchronized by an external clock. The sender uses *lockfile* to transmit a sequence of bytes, each one encoded with a Hamming code for reliability. The receiver tries to access *lockfile* at a high rate, with many attempts per Δt. Start with Δt at 10 sec and determine the error rate in the underlying data stream (after the Hamming code bits have been used to recover from errors and are stripped off) Then gradually reduce Δt by 100 msec each time and plot the error rate and bandwidth as functions of Δt.

47. In this chapter we have looked at operating systems security and ignored a major category of security problems that are not really related to operating systems, namely network security issues. In particular, we did not tackle viruses and worms much because doing so would have required another chapter and this book is big enough as is. Your assignment here is to write a 1-page report on computer viruses. Discuss the different kinds of them, how they do their work, how they spread, and how antivirus software attempts to track them down.